

DL Gen AI - Week - 3

L1

Intro to Vision Modelling

- For image data $\rightarrow$ CNNs (convolutional neural networks) handles 2D data efficiently, w/o losing the spatial info
- an image looks like a large grid (matrix) of numerical data. Each entry corresponds to a pixel (smallest unit).
- Binary img. $\rightarrow$ contains only 0 or 1 values.
 $\downarrow$ $\downarrow$
B W
- Grayscale $\rightarrow$ full shades of gray (0-255)
- RGB $\rightarrow$ 3 channels R, G, B, each pixel has 3 8 bit values.
 $\swarrow$ $\searrow$
red green blue
- RGB
img.

 $\rightarrow$ looks like $(3, 224, 224)$
 $\downarrow$
channels
- some CV tasks $\rightarrow$ img. classification, obj. detection, obj. recognition, semantic segmentation
- apply MLP to img. classification

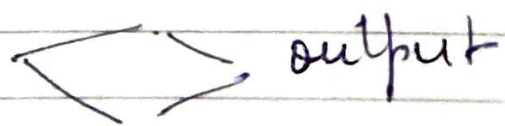
28 x 28
784

flattening



784 x 1

MLP



- this process loses the spatial info., also it leads to massive inc. in the no. of params. Thus, leading to slow training & massive memory usage. Flattening also leads to destruction of "semantics" of features.
- Further, it lacks translaⁿ invariance. MLP is not robust to an object's posⁿ. The problem is model learns specific wts. for features at specific loca^{ns}.

L2. From MLP to Convolution

Input: X ; hidden repres. H

X_{ij} be the pixels, H_{ij} corresp. hidden unit

FC layer req. a 4th order W (wt. tensor) to connect every input pixel to every hidden unit.

$$H_{ij} = \sum_{k,l} W_{ijkl} X_{kl}$$

So, it becomes very complex.

- 1. Translaⁿ Invariance.

Our pattern detector should work regardless of the posⁿ.

It means wt. should not depend on abs. pixel posⁿ (i, j) , but relative offset $(k-i, l-j)$

$$W_{i,j,k,l} \rightarrow V_{k-i, l-j}$$

$$H_{i,j} = \sum_{a,b} V_{a,b} X_{i+a, j+b}$$

→ Now this is 'your convoluⁿ'. Now the no. of params is only tied to the size of filter V .

- 2. Locality → info. from very far away is irrelevant.

We will now apply dist. constraint of filter wts. $V_{a,b}$.

$$H_{i,j} = \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} V_{a,b} X_{i+a, j+b}$$

→ this eq. defines the convoluⁿ layer. The learnable tensor V is c'd convoluⁿ kernel. It is now indep. of input img. size.

- By maths, convoluⁿ is,

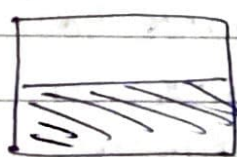
$$(V * X)_{i,j} = \sum_{a,b} V_{a,b} X_{i-a, j-b}$$

- The term "convolu" in DL has bit diff. defn, it is known as cross-correlation. It does n't matter much for us.
- Input $X = \{ht, width, channels\}$
- $H \rightarrow$ feature maps (activa")
- output

$$H_{i,j,C_0} = \sum_{a,b} \sum_{c_i,c_0} W_{a,b,c_i,c_0} X_{i+a,j+b,c_i} + bias_{c_0}$$

- The "convolu" layer is the building block of CNN. The "convolu" opera" here.
 $\rightarrow$ element-wise multiplica" and then add.

L3. Filters, Feature Maps & Receptive Fields



$$\begin{bmatrix} 30 & - & - & 30 \\ 30 & & & \\ 30 & - & - & 30 \\ 0 & - & - & 0 \\ 0 & - & - & 0 \\ 0 & - & - & 0 \end{bmatrix}_{6 \times 6}$$

edge is present here.

filter (3x3)

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$



$$\begin{bmatrix} 0 & - & - & 0 \\ 90 & - & - & 90 \\ 90 & - & - & 90 \\ 0 & 0 & - & 0 \end{bmatrix}_{4 \times 4}$$

4x4

- The filter will check if there is any sharp Δ from a high to low value.
 $\hookrightarrow$ Prewitt filter $\rightarrow$ used in traditional edge detection filter.
 $\hookrightarrow$ in CNN, filters are learnt.

- Process

1. filter wts. are randomly initialized.
 $W \sim N(0, \sigma^2)$
2. network has a predicⁿ $\hat{y} = f_W(x)$
3. got the error $L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$
4. do backpropagaⁿ $W \leftarrow W - \eta \frac{\partial L}{\partial W}$

do this iteratively to reach convergence

- Feature map $\rightarrow$ it is the output of 1 conv. filter applied to the input.
 activaⁿ map $\rightarrow$ it is the 2D slice in the output 3D volume.
 $\hookrightarrow$ for eg. for colours, edges, or green patches, etc.

- The receptive field is the size of seg. in the original image that affect its activaⁿ.

- _/_/_
- As we stack more & more convolutional layers, the receptive field grows. This helps to build a network of hierarchy of features. Edges $\rightarrow$ contours $\rightarrow$ small obj. $\rightarrow$ entire objects.

24. CNN: Hyperparameters.

- Kernel size (filter size) is the dimensions ($h \times w$) of the convolution window that slides over the input data. It determines the size of the receptive field.
- $3 \times 3 \rightarrow$ most commonly used.
- $5 \times 5, 7 \times 7 \rightarrow$ used to capture larger spatial patterns, often used initially.
- smaller kernels learn fine-grained features. While larger kernels capture the global features but at the cost of high computational cost. $\rightarrow$ tradeoff.
- A conv of 1×1 kernel $\rightarrow$ powerful technique known as Network in network layer. $\rightarrow$ used in dimensionality reducⁿ.

Eg. $56 \times 56 \times 192$

32 filter of size $1 \times 1 \times 192 \rightarrow$

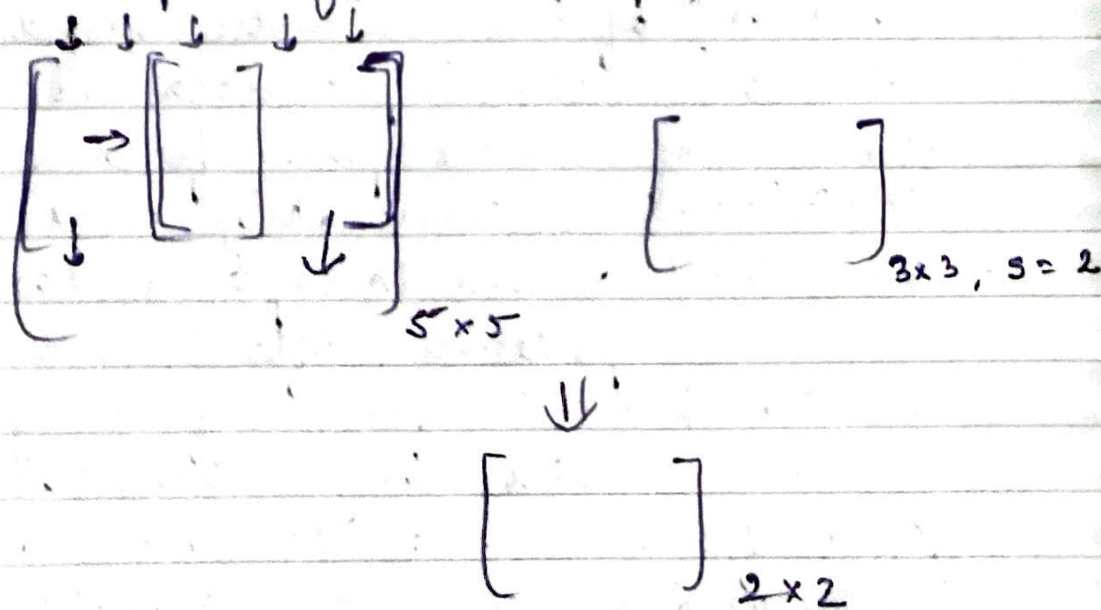
$\downarrow$ output

$56 \times 56 \times 32$

it aggregates the info. across the channels.

Eg. you add up non-linearity (like ReLU) so that networks can learn complex relationships between channels.

- **Stride** $\rightarrow$ step size, no. of pixels the conv. kernel moves over the input feature map at a time. Default, $S=1$, shift of 1 pixel at a time. This process leads to downsampling, if you use $S > 1$, then it reduces the $(h \times w)$ dimensions of the output feature map.



- $$\text{Output size} = \left(\frac{\text{Input size} - \text{kernel}}{\text{Stride}} \right) + 1$$

- **Padding** is the process of adding extra pixels ($P=0$), around the border of an input feature map. It helps you get better context at the edges.
Same as valid (0) padding

- size of output is controlled by ,
- K (kernel size)
 - S (stride)
 - P (padding)
- controls the output size

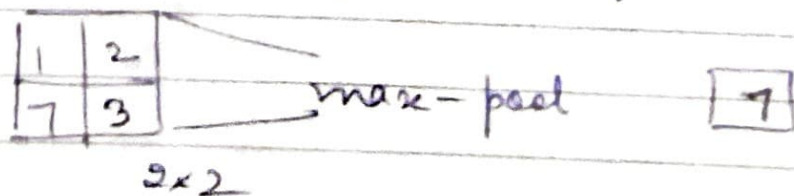
$$W_{out} = \frac{W - K + 2P}{S} + 1$$

$$H_{out} = \frac{H - K + 2P}{S} + 1$$

C_{out} = no. of convolution filters

LS CNN: Network layers

- A pooling layer performs non-linear downsampling to reduce the spatial dim. of feature maps. No wts., just reduces the receptive field. It helps in small translational invariance
- Max pool → selects max from each patch
↳ captures prominent info.
- Avg pool → computes the avg., smoothens the feature representation.



→ c/d internal covariance shift

1	2
7	3

→ avg.

3.25

$$\frac{3 \cdot 25}{4}$$

also, min-pool is possible.

- During training, distribution of each layer's inputs Δ s as the params are updated. This slows training & makes networks highly sensitive to wt. initialization.
- Batch Norm → technique to standardize the inputs to a layer for each mini-batch.
 - Benefits → higher η , reduces sensitivity to wts. initialization, acts as a regularizer (doesn't require a dropout)

- actual $B = \{x_1, \dots, x_m\}$

1. calc mean → $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$

2. " var → $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$

3. Normalize → $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$

4. scale & shift → $y_i = \gamma \hat{x}_i + \beta$

$\downarrow$ $\downarrow$
 scale shift
 can be learnt

- BN is applied per channel across batch & spatial dimensions. It is done for a feature map of shape (N, C, H, W)
- For training, use batch stats, for inference use running statistics.

so, for single img. $\mu_B = x_i$, $\sigma_B^2 = 0$

($\therefore$, we use populaⁿ statistics during inference)
 goal is to get stable, batch-size-independent stats for inference.

- Training

1. Forward pass $\Rightarrow y_i = \gamma \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$

2. update stats

$$\mu_{\text{run}} \leftarrow (1 - \alpha) \mu_{\text{run}} + \alpha \mu_B$$

running mean

momentum = 0.1
 same for variance.

we don't use it in the main formula.
 Just kept for updating.

after many batches it converges to batch stats

- Inference

$$y_i = \gamma \frac{x_i - \mu_{\text{run}}}{\sqrt{\sigma_{\text{run}}^2 + \epsilon}} + \beta$$

provides stability regardless of anything

- if $\alpha \uparrow$, then faster adapⁿ, more noise
 $\alpha \downarrow$, slow " less "

→ initializaⁿ → $\mu_u = 0, \sigma_u^2 = 1$

→ bias correctⁿ →

$$\hat{\mu}_u = \frac{\mu_u}{1 - (1 - \alpha)^t} \quad \begin{matrix} \nearrow \text{Training} \\ \text{step no.} \end{matrix}$$

- Dropout → powerful regularizaⁿ technique to prevent overfitting in NN.

→ each step, it sets a fracⁿ of neuron activaⁿ to 0.

→ forces the network to learn more robust & redundant features.

→ also, prevents co-adaptaⁿ (dependence of neighbourhood ↓)

→ only used during training.

- In training,

1. random masking

2. scale → remaining by $\frac{1}{1-p}$

$$y_i = \begin{cases} \frac{x_i}{1-p} & (1-p) \\ 0 & p \end{cases}$$

- In inference

1. no masking

2. no scaling

- _/_/_
- The scaling in training mode makes sure that the expected output remains the same.

$$E[y_i^{inf}] = E[y_i^{train}] = x_i$$

L6. Architecture & Data Augmentaⁿ

- CNN has layers of $\rightarrow$ convoluⁿ, activaⁿ, pooling layers & fully connected layer for classificaⁿ.
 $\hookrightarrow$ mostly softmax.
 - the convoluⁿ base (Conv $\rightarrow$ ReLU $\rightarrow$ Pool) acts as the feature extractor.
 - Flatten $\rightarrow$ it is the bridge btw. convoluⁿ layer & classificaⁿ layer.
 - Data Augmentaⁿ $\rightarrow$ process to artificially inc. the size & diversity of training dataset.
 1. It prevents overfitting & learning bias on training data.
 2. It teach invariance
 3. helps to combat data scarcity.
- $\rightarrow$ some common techniques — random rotaⁿ, flips, scaling, zooming, random cropping & color jitter.
- we will see these Fashion MNIST dataset.

- 60k training & 10k test image (28x28)
10 categories.

- filter $\Rightarrow 32 \rightarrow 64 \rightarrow 128$, 3 conv. layers
3x3 kernels

2x2 max pool

FC

dropout

output layer process

1x28x28

32x14x14

64x7x7

128x3x3

1152

128

10

- around total params $\rightarrow 100k$.

L7

Convolution from Scratch

$$(I \times K)(i, j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m, n)$$

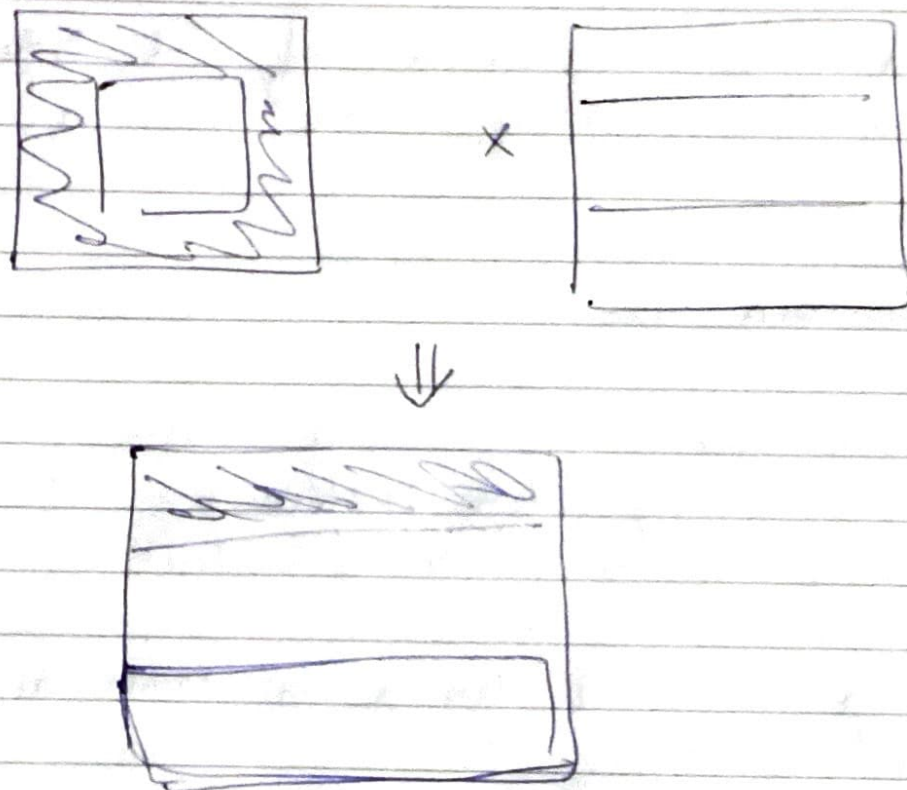
maneu of a cross-correlation

- key things -

1. local receptive field
2. learnable filters / kernel
3. feature detection
4. "activation" maps

- main HPTs $\rightarrow$ filter size, stride, padding,
no. of filters

- we can create custom kernels that helps to detect the edges clearly.
- convoluⁿ requires 2 'for' loops
- code customized for 2D convoluⁿ from scratch.



18

Pooling from scratch

- max
- avg.
- global pool $\rightarrow$ reduces each feature map to a single value (max/avg), used before FC layer.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

4x4

stride
= 2

max

pool

2x2

6	8
14	16

2x2

- manual implementaⁿ of all types of pools.
 ↳ then display their results.

L9. Convoluⁿ in PyTorch

- these things are quite slow if you do them using 'for' loop.
- define entire "2D convoluⁿ" in PyTorch
- to count the params.
 $\text{sum}(p.\text{numell})$ for p in $\text{conv-layer.parameters}()$
- wt. size $\rightarrow$
 $= \text{in-cha.} \times \text{out-cha.} \times \text{kernel size} \times \text{kernel size}$

L10 Pooling in PyTorch

- it works as -
 $\text{nn.MaxPool2D}(\text{kernel-size} = 2, \text{stride} = 2)$
 $\text{nn.AvgPool2D}(\text{" "})$
 create their instance
- for global pooling

nn. AdaptiveMaxPool2D(1)

nn. AdaptiveAvgPool2D(1)

it helps you to define your output dimensions.
helps also in flattening before the FC layer.

L11 CNN Image Classification - Simple CNN

- 1st conv. block

in-channel

out-channel

kernel size

stride

padding

↓ ReLU

↓ Max pool

- 2nd conv. block → 3rd conv. block

↓

FC1 → ReLU

↓

Dropout → FC2

- define the forward pass through the network.

- we can define a customized funcⁿ to visualize the intermediate feature maps for better understanding.

L12 Data Preprocessing & Dataloaders

- convert the img. to tensor & then normalize the dataset.
- prepare the train & test dataset in a dataloader.
- visualize some samples from the dataset, so that everything is coherent with each other.

L13 Model Training

- criterion $\rightarrow$ cross entropy loss
 $\hookrightarrow$ std. loss for multi-class classification
- Adam optimizer
- for each pass, update your wts. in the network